# DAY 3 LEARNING NOTES

Python Data Types & Type Conversion
DSA: Arrays

**Date:** 9 November 2026 **Study time:** 120 minutes

**Learning objectives**
Identify Python data types; explain mutable and immutable objects; convert values; understand input conversion; create and traverse lists; explain array operations and basic time complexity.

## PART 1 — PYTHON DATA TYPES

### 1. What is a data type?

A data type describes the kind of value a variable holds. Python uses dynamic typing: it determines a variable's type from the value assigned, so you do not need to declare the type explicitly.

```
name = "Suraj"\nage = 24\nheight = 5.5\nis_student = True\n\nprint(type(name)) # str\nprint(type(age))
# int\nprint(type(height)) # float\nprint(type(is_student)) # bool
```

### 2. Common built-in data types

| Type | Purpose | Example |
|---|---|---|
| int | Whole numbers | 25, -10 |
| float | Decimal numbers | 5.5, 99.99 |
| complex | Real and imaginary parts | 3 + 4j |
| str | Text | 'Python' |
| bool | Truth values | True, False |
| list | Ordered, mutable collection | [10, 20, 30] |
| tuple | Ordered, immutable collection | (10, 20, 30) |
| set | Unique values | {10, 20, 30} |
| dict | Key-value pairs | {'name': 'Suraj'} |
| NoneType | Absence of a value | None |

### 3. Numeric, string and Boolean types

**int** stores whole numbers; **float** stores decimal numbers; **complex** stores real and imaginary parts. A **string** is a sequence of characters in quotes. A **Boolean** is either True or False.

```
number = 3 + 4j\nprint(number.real) # 3.0\nprint(number.imag) # 4.0\n\nname = "Suraj"\nprint(name[0]) #
S\nprint(name[-1]) # j\nprint(10 > 5) # True
```

String indexing starts at 0. Index -1 refers to the last character. Strings are immutable: an individual character cannot be replaced directly.

### 4. Mutable vs immutable objects

Mutable objects can be changed after creation. Immutable objects cannot be changed in place.

| Mutable | Immutable |
|---|---|
| list | int |
| dict | float |
| set | str |

| | |
|---|---|
| — | tuple |
| — | bool |

```
numbers = [10, 20, 30]\nnumbers[0] = 100\nprint(numbers) # [100, 20, 30]\n\nname = "Suraj"\n# name[0] = "R" # TypeError
```

### 5. Type conversion

Type conversion changes a value from one data type to another. **Implicit conversion** happens automatically; **explicit conversion** is requested by the programmer.

```
# Implicit conversion
a = 10
b = 5.5
result = a + b
print(result) # 15.5
print(type(result)) # float

# Explicit conversion
age = int("24")
price = float("99.50")
marks_text = str(85)
```

Common conversion functions: int(), float(), str(), bool(), list(), tuple() and set(). Invalid conversions can raise ValueError; for example, int('hello').

### 6. Important: input() returns a string

Even if the user types digits, input() returns text. Convert it before arithmetic.

```
# Incorrect for numeric addition
a = input("First number: ")
b = input("Second number: ")
print(a + b) # "10" + "20" becomes "1020"

# Correct
a = int(input("First number: "))
b = int(input("Second number: "))
print("Addition:", a + b) # 30

# For decimal input
amount = float(input("Enter amount: "))
```

## PART 2 — DSA: ARRAYS

### 7. What is an array?

An array is a linear data structure that stores elements in indexed positions. Traditional arrays commonly store elements of the same type in contiguous memory. In beginner Python, lists are commonly used for array-style exercises. Python lists can contain mixed types and can grow or shrink dynamically.

**Example:** marks = [80, 85, 90, 75]. Index 0 contains 80; index 1 contains 85; index 2 contains 90; index 3 contains 75.

### 8. Creating, indexing and traversing a list

```
numbers = [10, 20, 30, 40, 50]

print(numbers[0]) # 10
print(numbers[2]) # 30
print(numbers[-1]) # 50

# Traverse each element
for num in numbers:
print(num)

# Traverse using indexes
for i in range(len(numbers)):
print("Index:", i, "Value:", numbers[i])
```

Remember: indexing starts at 0; the last valid index is n - 1. Accessing an invalid index raises IndexError.

### 9. Basic list / array operations

| Operation | Python example | Meaning |
|---|---|---|
| Access | numbers[0] | Read element at index 0 |
| Update | numbers[0] = 100 | Replace an element |
| Insert | numbers.insert(1, 15) | Insert at index 1 |
| Append | numbers.append(60) | Add at the end |
| Remove | numbers.remove(30) | Remove value 30 |
| Pop | numbers.pop() | Remove and return last element |
| Length | len(numbers) | Get number of elements |

### 10. Time complexity of common operations

Time complexity describes how an operation's running time grows as the input size n increases.

| Operation | Typical complexity | Reason |
|---|---|---|
| Access by index | O(1) | Direct access |
| Traversal | O(n) | Visit each element |
| Linear search | O(n) | May inspect every element |
| Update by index | O(1) | Direct replacement |
| Append to Python list | O(1) amortized | Occasional resizing |
| Insert at beginning | O(n) | Other elements shift |
| Delete from beginning | O(n) | Other elements shift |

O(1) means constant-time growth. O(n) means work grows proportionally with the number of elements. Python list append is O(1) amortized, not necessarily O(1) for every individual append.

# PART 3 — PRACTICE WORKSHEET

**Python assignments**

1. Create variables of type int, float, str, bool and complex. Print their values and types.
2. Convert the string '250' into an integer.
3. Take two numbers from the user and calculate sum, difference and multiplication.
4. Take a student's name and marks as input. Display both with their data types.
5. Demonstrate mutable vs immutable objects.

**DSA assignments**

6. Create a list containing five numbers and print it.
7. Print the first and last elements of a list.
8. Find the sum of all elements in a list.
9. Find the largest element without using max().
10. Count elements without using len().

### Guided examples for assignments 8–10

```
numbers = [10, 20, 30, 40, 50]

# 8. Sum (accumulator)
total = 0
for num in numbers:
total = total + num
print("Sum:", total) # 150

# 9. Largest (tracking variable)
largest = numbers[0]
for num in numbers:
if num > largest:
largest = num
print("Largest:", largest) # 50

# 10. Count (counter)
count = 0
for num in numbers:
count = count + 1
print("Count:", count) # 5
```

Problem-solving ideas: **total** remembers the running sum; **largest** stores the biggest value found so far; **count** increases once for every visited element.

**Independent challenge**

Given numbers = [12, 5, 27, 9, 18], write one program to calculate the sum, find the largest value and count the elements. Expected output: Sum: 71; Largest: 27; Count: 5.

## PART 4 — QUICK REVISION

- Common types: int, float, complex, str, bool, list, tuple, set, dict and NoneType.
- Lists, dictionaries and sets are mutable. Strings, tuples and numeric values are immutable.
- input() returns str; use int() or float() for numeric calculations.
- Python list indexes start at 0.
- Traversal and linear search are O(n); access by index is O(1).
- Use an accumulator for a sum, a tracking variable for a maximum, and a counter to count elements.

**Your Day 3 result**

Python assignments: 5/5 | DSA assignments: 5/5
Assessment: 6/6 (100%) | Difficulty: Moderate
Independent coding: Python assignments completed independently; further practice needed for the last three DSA assignments.

**Next lesson:** Day 4 — Python Operators and Expressions + DSA Array Operations.